

Étude de l'algorithme PageRank : puissance itérée, marche aléatoire et distances pondérées

1 Introduction

Le problème 2 du projet demande d'étudier PageRank comme chaîne de Markov sur un graphe orienté, de calculer sa distribution stationnaire par puissance itérée, puis de comparer cette référence à une estimation obtenue par simulation de marche aléatoire pour des longueurs de trajectoires $10^3, 10^4, 10^5, 10^6, 10^7$. Il est également demandé d'introduire une distance entre permutations afin de comparer les classements induits par les deux méthodes, avec en bonus une application sur un réseau réel.

L'idée probabiliste de PageRank est simple : un utilisateur se déplace d'une page vers l'une de ses sorties, mais garde une petite probabilité de téléportation vers les autres pages pour éviter les blocages dus aux nœuds puits ou aux composantes non communicantes. Si G désigne la matrice de navigation normalisée du graphe et Δ la matrice de téléportation uniforme hors diagonale, on travaille avec la matrice de transition

$$P = \alpha G + (1 - \alpha)\Delta,$$

avec $\alpha = 0.85$. Cette correction est essentielle : Elle assure que la chaîne soit irréductible, mais aussi apériodique. En effet, comme tous les sites sont reliés, on peut faire chaque chemin en 2 ou 3 étapes. Comme $\text{pgcd}(2, 3) = 1$, la période vaut 1.

On constate plusieurs phénomènes intéressants. D'une part, la marche aléatoire reproduit bien le PageRank exact dès que le nombre de pas devient grand. D'autre part, les distances globales classiques (Spearman ou Kendall) peuvent rester assez élevées alors même que les sommets les plus importants sont déjà correctement identifiés. On va donc chercher à utiliser des métriques plus adaptées à l'affichage de sites internet sur Google.

2 Modélisation et méthodes

2.1 Construction du PageRank

On considère d'abord un graphe orienté aléatoire de type Erdős-Rényi de taille $n = 1000$, où chaque arête orientée (i, j) , $i \neq j$, est présente avec probabilité $p = 0.01$. Dans l'expérience produite par le notebook, le graphe contient 10128 arêtes et aucun nœud sans lien sortant. Cela pourrait toutefois arriver, ce qui serait compenser par l'ajout de la matrice Δ . La matrice G est obtenue en normalisant chaque ligne de la matrice d'adjacence, tandis que Δ redistribue la masse uniformément sur tous les autres sommets.

On considère qu'initialement, l'utilisateur peut être uniformément sur chaque site (ce qui n'a pas d'importance dans la limite), puis π_n est défini par une relation de récurrence. On note π_n

verticalement pour les calculs en python :

$$\pi_0 = \frac{1}{n} \begin{pmatrix} 1 \\ \vdots \\ 1 \end{pmatrix} \quad ; \quad \pi_{n+1} = P^\top \pi_n$$

La loi stationnaire π est alors définie par

$$\pi = P^\top \pi, \quad \sum_{i=1}^n \pi_i = 1.$$

On commence par estimer cette loi par puissance itérée. À chaque itération, on remplace $\pi^{(m)}$ par $P^\top \pi^{(m)}$ jusqu'à ce que la distance ℓ^1 entre deux itérés successifs soit inférieure à 10^{-9} (de manière arbitraire). L'intérêt de cette méthode est double : elle est très simple à implémenter et elle correspond exactement à la définition initiale de π dans le cadre des chaînes de Markov.

2.2 Estimation par marche aléatoire

La seconde approche ne calcule pas explicitement la loi stationnaire. On simule une trajectoire $X_0, X_1, \dots, X_{T-1}$ de la chaîne de Markov donnée par P et l'on estime la probabilité stationnaire par les fréquences empiriques de visite

$$\hat{\pi}_i(T) = \frac{1}{T} \sum_{t=0}^{T-1} \mathbf{1}_{\{X_t=i\}}.$$

P est bien récurrente donc le théorème ergodique s'applique, et avec $f = \mathbf{1}_{\{i\}}$ on a

$$\hat{\pi}_i(T) \xrightarrow{T \rightarrow +\infty} \pi_i$$

L'implémentation de l'échantillonnage d'un saut a été rendu efficace en pré-calculant, pour chaque ligne de P , sa fonction de répartition cumulée puis en utilisant une recherche dichotomique via `searchsorted`. Ceci est plus rapide que de tirer un sommet à chaque fois en utilisant le vecteur des probabilités explicitement.

Le coût des deux approches n'est cependant pas du même type. La puissance itérée demande de manipuler une matrice de transition, donc de la mémoire et des produits matrice-vecteur. La marche aléatoire, elle, est extrêmement légère en mémoire mais au vu son aspect aléatoire on s'attend à ce que son erreur décroisse plus lentement.

2.3 Comparer des classements

Le vecteur stationnaire induit un classement des sommets par ordre décroissant d'importance. Si σ_m désigne la permutation associée à la puissance itérée et σ_s^n celle obtenue avec une marche de longueur 10^n , il faut choisir une distance entre permutations.

On étudie d'abord deux distances classiques :

- la distance de Spearman (ou *footrule*), qui somme les écarts absolus de rang ;
- la distance de Kendall, qui compte les paires dont l'ordre relatif est inversé.

On essaye aussi la précision à k ($P@K$), distance qui mesure le nombre de sites en commun avant le k -ième rang des deux permutations. De cette manière on peut se concentrer sur les premiers sites, qui peuvent nous intéresser plus que la suite.

Finalement, on introduit des versions pondérées de Spearman et de Kendall, où la contribution d'un sommet ou d'une paire est multipliée par un poids décroissant avec le rang de référence. Trois familles ont été testées : $1/i$, $1/i^2$ et $1/\log_2(i+1)$. Le schéma $1/i^2$ concentre presque toute l'attention sur les toutes premières positions ; le schéma logarithmique est plus progressif ; la pondération harmonique $1/i$ offre un compromis. Cette idée semble être le point le plus intéressant du travail, car elle reconnecte la métrique à l'objectif applicatif réel : en pratique, on se soucie beaucoup plus des dix ou cinquante premiers sommets que des derniers. Toutefois, elle accorde une importance à l'ordre des sites, au contraire de P@K.

3 Résultats sur le graphe aléatoire

3.1 Convergence de la puissance itérée

Sur le graphe aléatoire de taille 1000, la puissance itérée converge en seulement 17 itérations pour atteindre une différence finale de l'ordre de 3×10^{-10} en norme ℓ^1 . C'est un résultat très satisfaisant : malgré la taille du graphe, la chaîne modifiée par téléportation mélange rapidement et la référence numérique est obtenue à coût modéré.

On constate sur la figure 1 une convergence géométrique assez parfaite, sur les 17 itérations toutes sont alignées quand tracées dans l'échelle log. Ceci est cohérent avec l'évolution géométrique induite par l'algorithme.

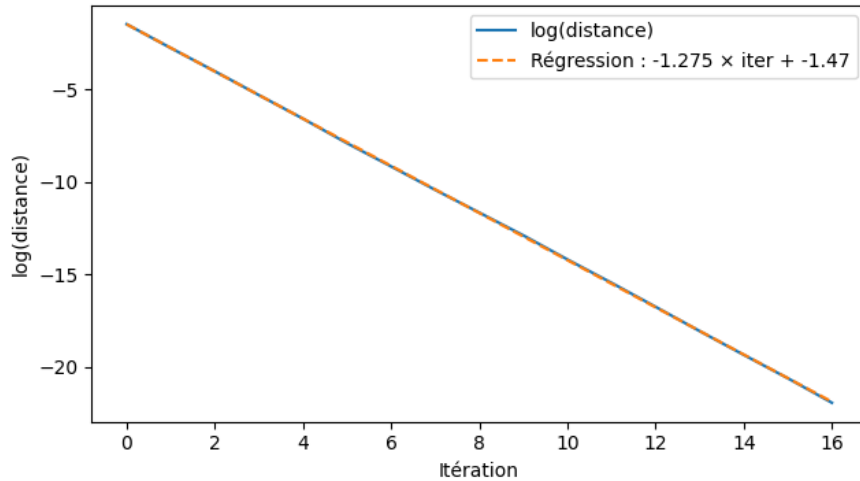


FIGURE 1 – Convergence de π_n via la méthode de la puissance itérée

3.2 Approximation de π par marche aléatoire

L'erreur moyenne composante par composante entre $\hat{\pi}(T)$ et π décroît régulièrement. On constate sur la figure 2 que la décroissance est bien polynomiale. On a alors une vitesse de décroissance en x^{-a} , avec

$$\begin{aligned} a &= -1.1591/\ln 10 \\ &= -0.503 \end{aligned}$$

car on a tracé le plot en base 10. On a bien une convergence en racine carré, ce qui est parfaitement cohérent avec la vitesse de convergence des moyennes empiriques. On peut en fait montrer que le biais est de l'ordre de $O(\frac{1}{n})$, et la variance en $O(\frac{1}{\sqrt{n}})$, d'où le résultat. On peut même aller plus loin avec une forme de TCL qui montre que la distribution des erreurs tend vers une gaussienne, ce qui est illustré sur la figure 3

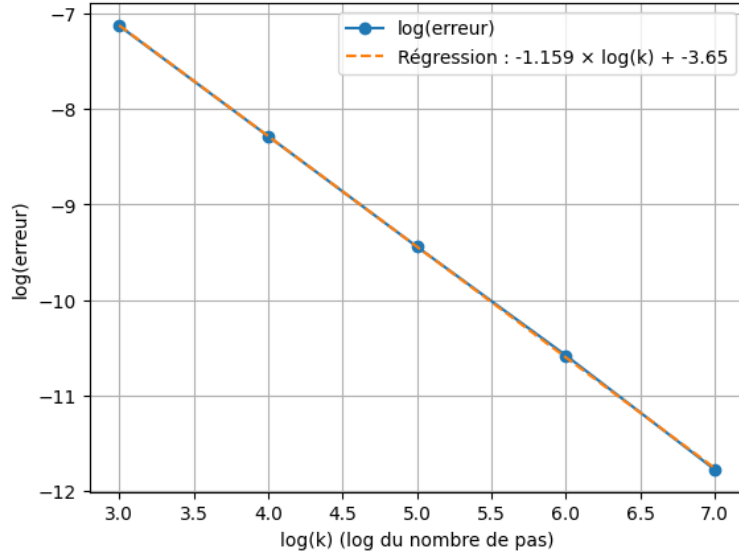


FIGURE 2 – Convergence de $\hat{\pi}$ via la méthode de la marche aléatoire

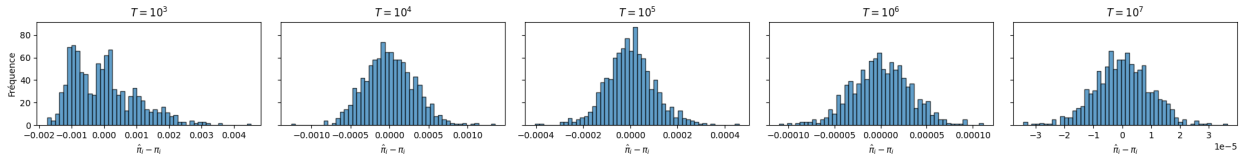


FIGURE 3 – Distribution des erreurs de π_s pour différents n

Ainsi, la marche aléatoire est statistiquement correcte, mais elle paie en longueur de trajectoire ce qu'elle économise en stockage matriciel.

3.3 Distances sur les classements

On pourra consulter le notebook pour les détails de l'implémentation des distances. Toutes ont été normalisées pour valoir au maximum 1, et 0 si les permutations sont égales. Le tableau 1 synthétise les principaux résultats obtenus sur le graphe aléatoire.

T	Spearman	Kendall	P@10	P@50	$wS_{1/r}$	wS_{1/r^2}	$wK_{1/r}$
10^3	0.5744	0.4222	0.10	0.14	0.3278	0.1445	0.3034
10^4	0.3771	0.2693	0.20	0.36	0.1840	0.1167	0.1638
10^5	0.1464	0.1044	0.60	0.74	0.0420	0.0041	0.0329
10^6	0.0489	0.0343	0.90	0.88	0.0132	0.0020	0.0091
10^7	0.0157	0.0106	0.90	0.98	0.0041	0.0001	0.0026

TABLE 1 – Comparaison entre le classement obtenu par puissance itérée et celui issu de la marche aléatoire sur le graphe Erdős–Rényi. wS et wK désignent respectivement Spearman pondéré et Kendall pondéré.

Plusieurs commentaires s'imposent. D'abord, Spearman et Kendall décroissent bien vers zéro, ce qui confirme la cohérence globale des deux classements. Les deux ne représentent pas la même chose, Spearman représentant la position, et Kendall la conservation de l'ordre. Il m'est compliqué d'évaluer laquelle est la plus pertinente dans le cas d'un moteur de recherche (notamment dans le cadre de la monétisation). On utilise donc les deux.

Mais leur niveau reste relativement élevé pour des valeurs intermédiaires de T : par exemple, à 10^5 pas, la distance de Spearman vaut encore 0.1464. Si l'on regardait uniquement cette métrique, on pourrait conclure que le classement reste assez éloigné de la référence. Ces distances sont en effet pertinentes si l'on souhaite comparer globalement deux classements, mais elles présentent une limite importante dans le contexte de PageRank : elles traitent de la même manière une erreur au sommet du classement et une erreur tout en bas.

Or ce diagnostic est trop pessimiste dès qu'on s'intéresse surtout au sommet de la liste. En effet, on peut regarder une autre métrique, la précision à k . On constate alors que la précision dans le top-10 est déjà de 0.60 pour 10^5 pas et atteint 0.90 à partir de 10^6 .

Cependant cette mesure de précision oublie complètement l'ordre, et si les 10 premiers sont complètement inversés elle donnera un résultat parfait. On utilise donc une nouvelle forme de distance, pondérée. De cette manière on garde l'importance de la position, tout en permettant de donner plus de valeur aux premiers résultats. Les distances pondérées chutent alors beaucoup plus vite : le Spearman pondéré avec poids $1/r^2$ tombe à 0.0041 dès 10^5 pas, puis à 1.18×10^{-4} pour 10^7 . Cela signifie que les erreurs résiduelles portent essentiellement sur des sommets de rang moyen ou faible, tandis que la tête du classement est déjà très bien capturée.

Autrement dit, les pondérations modifient l'interprétation du résultat. Les distances uniformes répondent à la question "les deux permutations sont-elles globalement proches?", alors que les distances pondérées répondent plutôt à "les sommets les plus importants sont-ils rangés correctement?". Pour une application de type moteur de recherche ou recommandation, la seconde question semble être la bonne.

4 Passage aux graphes réels

On a ensuite appliqué la même démarche à un réseau social et à un graphe de citations. On teste dans le notebook deux bases réelles : le réseau **wiki-Vote** de SNAP et le graphe de citations **Cit-HepTh**. Cette partie est importante car elle montre que les conclusions observées sur le graphe aléatoire ne sont pas un artefact d'un modèle trop simple. On a ici dû utiliser d'autres méthodes de calcul sans stocker explicitement la matrice du graphe, car celle-ci est assez *sparse*.

Sur **wiki-Vote** (7115 nœuds, 103689 arêtes), la puissance itérée converge en 26 itérations et l'implémentation est validée numériquement contre **networkx** avec une erreur maximale de 3.29×10^{-7} . Les performances comparées par la suite ne reposent donc pas sur une approximation douteuse de la référence. La marche aléatoire donne alors les résultats du tableau 2.

T	Spearman	Kendall	P@10	P@50	P@100	$wS_{1/r}$
10^4	0.3769	0.3024	0.70	0.68	0.76	0.0679
10^5	0.3444	0.2579	0.90	0.86	0.90	0.0428
10^6	0.3114	0.2315	1.00	0.98	0.97	0.0344
10^7	0.3009	0.2268	1.00	0.98	0.99	0.0322

TABLE 2 – Résultats obtenus sur le graphe **wiki-Vote**.

Le contraste entre distances globales et métriques focalisées sur le top est ici encore plus net. Même à 10^7 pas, les distances de Spearman et Kendall restent non négligeables, autour de 0.30 et 0.23. Pourtant, le top-10 est exactement récupéré et le top-100 est presque parfait. Cela suggère que les gros écarts de rang portent essentiellement sur une large masse de nœuds peu influents, vraisemblablement difficiles à ordonner finement parce que leurs scores de PageRank sont très proches. Dans cette situation, une distance uniforme punit sévèrement des erreurs qui sont en pratique peu importantes.

Le graphe de citations **Cit-HepTh** renforce encore cette lecture. Pour 27770 nœuds et 352807 arêtes, la puissance itérée sparse converge en 95 itérations. Les top articles obtenus semblent stables, et la marche aléatoire retrouve un top-10 exact dès 10^6 pas. Surtout, le Spearman global passe de 0.3849 à 0.0976 entre 10^5 et 10^7 pas, tandis que le Spearman pondéré $1/r$ descend déjà à 0.0089 pour 10^7 . Là encore, la hiérarchie des sommets dominants est beaucoup plus robuste que l'ordre complet des dizaines de milliers de nœuds.

5 Discussion

Le principal enseignement du projet est qu'il faut distinguer deux niveaux de comparaison. Si l'on veut approcher l'intégralité du vecteur stationnaire ou la permutation complète, la puissance itérée est clairement la meilleure solution dès que l'on peut faire des produits matrice-vecteur. Elle converge vite, donne une référence déterministe, et sa validation sur **wiki-Vote** montre qu'elle est numériquement fiable.

La marche aléatoire, à l'inverse, est une méthode statistique. Elle est attractive lorsque la matrice complète est trop coûteuse à stocker ou lorsque l'on peut échantillonner localement les voisins sortants sans matérialiser le graphe entier. Son défaut est sa lenteur statistique. Néanmoins, si l'objectif n'est pas le classement exhaustif mais seulement l'identification des sommets les plus influents, elle devient compétitive. C'est exactement ce que mettent en évidence les métriques pondérées.